

Amazon Q Handbook

Welcome to the Amazon Q Handbook — your guide to understanding and using Amazon Q for software modernization and transformation.

Table of Contents

1. [Amazon Q Overview](#)
 2. [Amazon Q Modernization \(Transformation\)](#)
 3. [Amazon Q IDE Integration](#)
 4. [Typical Use Cases](#)
 5. [Best Practices](#)
 6. [Common Issues](#)
 7. [Troubleshooting and Tips](#)
 8. [Resources and Further Reading](#)
 9. [FAQ](#)
-

Amazon Q Overview

What is Amazon Q?

Amazon Q is a **generative AI assistant** designed to accelerate software development and application modernization. It's built on **Amazon Bedrock**, using foundation models (FMs) to assist with:

- Coding, testing, upgrading, and troubleshooting
- Security scanning and infrastructure optimization

Key Capabilities

- **Generative AI Assistance:** Accelerates developer workflows
- **Developer Agents:** Execute intelligent tasks on behalf of devs
- **Enterprise Data Integration:** Works with your internal systems and documents
- **Security & Privacy:** Enterprise-grade compliance and governance

Product Offerings

Amazon Q Business

Provides AI support across enterprise knowledge bases and tools:

- Answers business questions using context-aware data
- Generates content and supports knowledge workers
- Integrates with apps like Slack, Jira, Confluence

Amazon Q Developer

Assists across the full development lifecycle:

- Writes, tests, debugs, and refactors code
- Supports modernization of legacy applications
- Automates cloud-native migrations and CI/CD setup

Amazon Q Services

- **BI (QuickSight):** Natural language dashboards and reports
- **Customer Support (Connect):** Real-time intent detection and responses
- **Supply Chain AI:** Actionable insights from AWS Supply Chain Data Lake

Pricing

Q Business:

- Lite – \$3/user/month
- Pro – \$20/user/month

Q Developer:

- Free Tier – Limited monthly use
- Pro Tier – \$19/month

Example Use Cases

- **Windows to Linux Migration:** Cuts migration time from months to days
- **VMWare to Cloud-Native:** Automates replatforming
- **Mainframe to Modern Stack:** Analyzes COBOL and accelerates migration

Amazon Q Modernization (Transformation)

Amazon Q Developer provides powerful transformation capabilities to help teams modernize legacy applications in .NET and Java. It automates large portions of the upgrade and refactoring process, reducing months of manual effort into days or weeks — while ensuring your application is compatible with modern, cross-platform, cloud-native standards.

Supported Platforms and Targets

Language	From	To
.NET	.NET Framework 3.5+, .NET Core 3.1, .NET 5+	.NET 8 (cross-platform)
Java	Java 8+	Java 17 or Java 21 (Spring Boot 3.2 compatible)
VMWare	VMWare	Cloud-Native Applications

Language	From	To
Mainframe	COBOL	Linux-compatible and Cloud-ready Applications

 Transformation Process Overview

1. Preparation

- Ensure your code builds successfully locally (using supported build tools).
- For .NET: Use **Visual Studio** (not VS Code).
- For Java: Use **VS Code**, **IntelliJ IDEA**, or **Eclipse (module-level only)**.

2. Analysis & Planning

- Amazon Q analyzes the codebase to identify:
 - Language and framework versions
 - Packages and dependencies
 - Compatibility issues and deprecated components
- Generates a **Transformation Plan** that outlines proposed changes.

3. Transformation Execution

- Language version upgrade (C# or Java)
- Package/library upgrades (e.g., NuGet, Spring Boot, JUnit)
- API replacements for deprecated or platform-specific code
- Code refactoring for Linux/cloud compatibility
- Test case validation (Java) and readiness report (for .NET)

4. Review & Acceptance

- A diff view and summary of changes is presented
- Developers can review and approve changes
- Manual intervention areas are highlighted (e.g., UI layers in .NET, legacy frameworks in Java)

 Web-Based Modernization Experience

Amazon Q also provides a **web-based modernization interface**, useful for enterprise or team-scale transformations:

1. **Sign in** using AWS IAM Identity Center credentials
 2. **Create a Workspace** to manage jobs
 3. **Connect Source Repositories** (GitHub, GitLab, CodeCommit, etc.)
 4. **Analyze and Approve Plan**
 5. **Execute Transformation** and download results
 6. **Test and Deploy** your modernized application
-

 Supported Application Types

.NET

- Console apps, class libraries, unit tests, Web APIs, WCF services
- Business logic in MVC and SPA apps
- *UI layers (WebForms, Razor Views) require manual updates*

Java

- Monolith Spring apps → Spring Boot microservices
- Java Swing applications (UI logic partially refactored)
- Embedded SQL apps (Oracle → PostgreSQL)
- Projects built with Maven or Gradle only

 Quotas & Limits

Tier	Max Lines/Job	Max Jobs/Month	Concurrent Jobs	Notes
.NET (Pro)	100,000	100,000 total	2/account	Visual Studio IDE or Web Experience
Java (Free)	1,000	2,000	1/user	Limited transformation
Java (Pro)	Higher	1,000+	1/user	More flexible, IAM Identity Center required

 Limitations

- Custom/legacy frameworks may need manual migration
- Complex SQL and Java reflection code may not auto-convert
- .NET UI components must be manually recreated
- IIS-specific configurations in .NET only partially supported
- Java transformations target only versions 17 or 21

 Real-World Use Cases

Scenario	Impact
Windows to Linux (for .NET)	Up to 40% licensing cost savings, reduced timeline from 6–8 months to a few days
VMWare to Cloud-Native	Months of work condensed into weeks
Mainframe (COBOL) to Cloud	Automatic documentation + >50% faster migration

 Subscription Requirements

- **.NET Transformation:** Requires **Amazon Q Developer Pro Tier**
- **Java Transformation:** Available in **Free Tier (limited)** and **Pro Tier** for higher limits

Amazon Q IDE Integration

Amazon Q provides deep integration with leading development environments to bring transformation, code generation, testing, and documentation directly into the developer’s workflow. These integrations help accelerate modernization efforts by allowing developers to invoke AI actions contextually within their IDEs.

 Supported IDEs

IDE	Supported Languages	Notes
Visual Studio	.NET	Required for .NET transformation; via AWS Toolkit
Visual Studio Code	Java	Full feature support including chatbot and transformations
IntelliJ IDEA	Java	Supported for Java transformation and assistant features

 Key IDE Features

Amazon Q offers a chatbot panel, right-click contextual menus, and inline actions for:

- Code transformation (Java and .NET)
- Code generation (**/dev**)
- Unit test creation (**/test**)
- Code review (**/review**)
- Documentation generation (**/doc**)
- SQL & language upgrades (**/transform** for Java)

 File-Level Actions

Developers can right-click files in the editor and invoke Amazon Q options such as:

- Upgrade Java version (e.g., Java 8 → Java 17/21)
- Refactor code for modern standards
- Generate tests or documentation
- Run code reviews inline

 Sidebar Chatbot Panel

A dedicated chat assistant available in VS Code and IntelliJ:

- Understands **@workspace** or specific file references
- Supports commands like `/dev`, `/test`, `/review`, `/doc`, and `/transform`
- Handles cross-file and dependency-aware logic

Example Prompts:

```
@workspace /transform upgrade Java 8 Spring Boot to Java 17
/dev write a REST API controller for user management
/test create unit tests for HelloWorld.java
```

 IDE Workflow: .NET Modernization

Amazon Q supports .NET transformation through the **Visual Studio IDE** using the **AWS Toolkit**.

Steps:

1. Install the AWS Toolkit in Visual Studio
2. Sign in using IAM Identity Center credentials
3. Open your .NET Framework solution (.sln)
4. Right-click → Amazon Q → “Transform to .NET 8”
5. Review the generated transformation plan
6. Apply the changes and test

 Important Notes:

- Visual Studio **is required** for .NET transformations
- Visual Studio Code is **not supported** for this purpose
- The Amazon Q Developer Web Experience can be used for .NET transformations but has its own limitations
- UI layers like WebForms or Razor Views require manual updates

 Feature Matrix by Language & IDE

Feature	Java (VS Code / IntelliJ)	.NET (Visual Studio)
Code Transformation	☒	☒
Code Generation (<code>/dev</code>)	☒	☒
Unit Test Generation (<code>/test</code>)	☒ (Java, Python only)	☐
Code Review (<code>/review</code>)	☒	☒
Documentation (<code>/doc</code>)	☒	☒
SQL Migration (Oracle → PostgreSQL)	☒	☐
UI Refactoring Assistance	Partial (Java Swing)	Manual (Razor/WebForms)

Typical Use Cases

Amazon Q is best used in scenarios where speed, scale, and modernization are critical. It helps teams upgrade legacy codebases, migrate applications to cloud-native platforms, and automate time-consuming development tasks.

.NET Use Cases

1. ASP.NET MVC 5 to ASP.NET Core MVC

- Transforms business logic and controller layers
- Refactors backend for .NET 8 compatibility
- UI layers (Razor Views) require manual updates

2. Windows to Linux Migration

- Converts Windows-hosted .NET applications to Linux-compatible .NET
- Replaces platform-specific APIs
- Identifies and documents Windows-only dependencies
- **Impact:** Up to 40% savings in licensing costs and significantly faster project timelines

3. IIS-Hosted App Transformation

- Supports apps with default IIS configurations
 - Prepares codebase for Kestrel or cross-platform hosting
 - Offers Linux readiness report
-

Java Use Cases

1. Java 8 to Java 17+ Upgrade

- Updates deprecated APIs and language constructs
- Upgrades libraries like Spring, Spring Boot, JUnit, and Mockito
- Adds compatibility with newer LTS releases

2. Java Swing Refactoring

- Transforms Java 8 Swing-based UI apps for Java 17+
- Replaces outdated methods with modern equivalents
- Improves compatibility with current JVM standards

3. Spring Boot Migration

- Upgrades Spring Boot 2.x to 3.2 (Java 17 compatible)
- Adjusts annotations, autowiring, and configurations
- Updates dependency versions

4. Embedded SQL Migration

- Converts inline Oracle SQL to PostgreSQL
 - Helps prepare apps for open-source database platforms
 - Notes complex SQL for manual review
-

Cross-Language / General Use Cases

1. Monolith to Microservices Prep

- Refactors tightly coupled logic into services
- Organizes code into modules with well-defined boundaries

2. Mainframe Modernization

- Analyzes millions of lines of COBOL
- Auto-generates documentation to explain legacy systems
- Cuts transformation time by 50% or more

3. Test Case Generation

- Uses `/test` to auto-generate unit tests for Java and Python
- Validates transformation by executing pre/post tests

4. Documentation and Onboarding

- Uses `/doc` to generate:
 - README files
 - API documentation
 - Architecture and onboarding guides

5. Code Review and Security Fixes

- `/review` highlights code smells, security issues, and anti-patterns
 - Ideal for pre-deployment checks or modernization audit
-

Best Practices

To get the best results from Amazon Q's transformation and development capabilities, follow these best practices across both .NET and Java modernization workflows.

☒ General Preparation

-  **Clean your codebase** before starting: remove unused files, fix compile-time issues, and validate your test suites
-  **Run local builds** and unit tests to confirm that your code is stable before uploading or transforming

-  **Use supported build tools:**
 - Java: Maven or Gradle
 - .NET: Visual Studio solution (.sln)
 -  **Use well-structured codebases:** modularize your components, keep file organization clear, and follow standard design patterns
-

Transformation Best Practices

-  **Start with non-UI components:**
 - Transform models, services, and APIs first
 - Leave UI layers (e.g., Razor Views or Swing UIs) for manual review
 -  **Apply transformations incrementally:**
 - Migrate small components and validate them before continuing
 -  **Use the Linux Readiness Report** (for .NET) to address system-level compatibility issues
 -  **Validate transformation with diff views** before merging the output into your repo
-

Prompting & Chatbot Usage

-  Use `@workspace` or reference specific files for better context when prompting the chatbot
 -  Keep prompts concise and task-specific:
 - ☒ `/dev generate a service to fetch orders by ID`
 - ☒ `/test create unit tests for OrdersController.java`
 - Use the `/review` action to catch missed issues before transformation or deployment
-

Testing & Validation

-  Ensure that existing tests pass after transformation
 - ☒ Add new unit tests (via `/test`) to validate critical paths
 -  Test on your target runtime/platform (e.g., .NET 8 on Linux, Java 17+ JVM)
-

Post-Transformation Manual Review

- UI layers require manual migration:
 - .NET: Razor Views, WebForms
 - Java: Swing components
 - **Security checks and configurations** (e.g., auth, CORS, HTTPS) should be reviewed manually
 - Confirm **dependency injection setups**, `Startup.cs`, or `application.yml/application.properties` are aligned with target frameworks
-

Documentation

- Use `/doc` to generate:
 - Project READMEs
 - API documentation
 - Onboarding or design overviews

- Always **review generated documentation** to fill in architecture-specific details
-

Common Issues

Despite Amazon Q's powerful capabilities, users may run into common issues during setup, transformation, or post-processing. Below is a list of common pitfalls and limitations to be aware of when using Amazon Q Developer.

Codebase & Structure

-  **Code doesn't build locally (when performing IDE-based transformations)** Amazon Q performs build verification before transformation. If your project fails to build, the process will halt. Ensure your build scripts, dependencies, and environment are working as expected.
 -  **Poorly structured codebase**
Disorganized or tightly coupled code can reduce transformation accuracy. Use modular components and follow standard architecture practices.
 -  **Project not detected or recognized**
 - **.NET:** Ensure a Visual Studio Solution File (`.sln`) exists and is in the root directory.
 - **Java:** Ensure `pom.xml` is in the root directory and at least one `.java` file is present.
 - **Java + VS Code:** The project must be opened such that `pom.xml` is in the workspace root.
 -  **Unsupported UI layers**
UI frameworks like Razor Views, WebForms (for .NET), and complex Java Swing interfaces are not automatically transformed. These require manual effort.
 -  **Unsupported or legacy dependencies**
.NET: Only Microsoft-authored NuGet packages are fully supported.
Java: Custom or legacy libraries may require manual adjustments post-transformation.
-

Configuration & Tooling

-  **Using Visual Studio Code for .NET**
Amazon Q transformation for .NET only works in Visual Studio (Windows) with the AWS Toolkit installed. VS Code is **not supported** for .NET modernization.
-  **IAM Identity Center not configured**
The web-based transformation experience requires IAM Identity Center (formerly AWS SSO). If not configured, users will be unable to authenticate.
-  **Authentication failures in unsupported regions**
The Amazon Q web experience currently only works in **us-east-1 (N. Virginia)** and **eu-central-1 (Frankfurt)**. Attempting to use it in other AWS regions may result in login or job execution failures.
-  **AWS Toolkit missing in Visual Studio**
Developers must install the latest AWS Toolkit extension to access Amazon Q transformation features in

Visual Studio.

Limits & Quotas

-  **Codebase too large**
.NET transformations are capped at **100,000 lines/month**.
Java Free Tier is limited to **1,000 lines/job** and **2,000 lines/month**. Upgrade to Pro Tier for larger projects.
-  **Concurrent job restrictions**
.NET: Only **2 concurrent jobs per AWS account**
Java (Free Tier): Only **1 job per user at a time**

Feature Limitations

-  **Java version constraints**
Ensure your project uses **Maven or Gradle** and compiles successfully.
-  **Lack of unit tests**
Java transformations rely on unit tests for validation. If tests are missing or incomplete, confidence in the output decreases.
-  **Framework incompatibility**
Projects using custom JNI code, heavy reflection, or proprietary frameworks may not be fully transformable. Manual refactoring may be required.
-  **Incorrect or incomplete transformations**
 - **.NET**: The web experience may skip certain layers (e.g., UI or config). A manual review of the output is recommended.
 - **Java**: Many backward-compatible patterns and libraries are left unchanged; developers should still review the code for modernization opportunities.

Performance Issues

-  **Slow performance on repositories with build errors (when performing web-based transformations)**
Amazon Q runs an **Agentic workflow** to resolve build errors automatically. Depending on the number and complexity of issues, this process may take **several minutes to multiple hours**.

Troubleshooting and Tips

This section focuses on practical solutions to help resolve transformation errors, configuration issues, and workflow friction when using Amazon Q Developer.

Quick Fix Checklist (Before You Begin)

- ☒ Run a clean local build using Visual Studio, Maven, or Gradle
- ☒ Ensure `pom.xml` (Java) or `.sln` (C#) is in the root folder
- ☒ Confirm IAM Identity Center login works (for web-based UI)
- ☒ Open Java projects in VS Code with `pom.xml` in the root
- ☒ Verify you're working in supported regions (`us-east-1`, `eu-central-1`)
- ☒ Check that project size does not exceed free/pro tier limits

 Common Problems & Fixes

Symptom	Resolution
Project not recognized	Ensure <code>.sln</code> or <code>pom.xml</code> is in the project root. For Java in VS Code, open the correct folder.
Can't sign in (web experience)	Use IAM Identity Center and a supported region
Partial or incomplete transformation	Manually review diff; UI layers may not be supported
Transformation very slow	Q may be resolving build errors (Agentic workflow); simplify if possible
"Transformation failed"	Run local build, fix compile-time errors, check dependencies
No output or unchanged files	Code may already be compatible; prompt for additional modernization
Toolkit options missing in Visual Studio	Install latest AWS Toolkit; restart the IDE

 Prompting Tips (In IDE Chatbot)

- Use `@workspace` for full context
- Target specific files for more control:
`/test write tests for OrderService.java`
`/review find issues in UserController.cs`
- Break large tasks into multiple smaller prompts
- Ask for summaries before applying large-scale changes

 Reviewing Output

After transformation:

- Use **Diff View** to inspect changes
- Read the **Transformation Summary**
- For .NET, check the **Linux Readiness Report** for manual follow-ups
- Always test the transformed app before deployment

 Pro Tip

Use this section as a **triage guide**—refer to it when something breaks, but avoid repeating it in every team sync. Let your team bookmark or print this as a "Ready to Transform" checklist.

Resources and Further Reading

- [Amazon Q Official Documentation](#)
 - [Amazon Q on AWS Blogs](#)
-

FAQ

1. Is Amazon Q free?

- Amazon Q offers both free and paid tiers. Basic access, such as using Q in the AWS Console or with some IDE integrations, may be available at no cost.
- However, advanced transformation capabilities — especially enterprise features like full legacy modernization — requires a subscription or be part of AWS enterprise support or professional services.

2. What programming languages are supported by Amazon Q?

Amazon Q currently supports:

- **Java** (legacy versions and modernization to modern Java frameworks)
- **.NET Framework** (modernization to .NET Core / .NET 6+)
- **JavaScript / TypeScript, Python**, and other common languages for code generation or assistance.

Support for other languages may be limited or under development.

3. Can I customize the output generated by Amazon Q?

Yes. While Amazon Q generates transformation suggestions based on best practices, developers can:

- Customize configuration files (when applicable)
- Accept or reject individual suggestions
- Modify the output code before committing it
- Integrate the output into CI/CD workflows for further validation
- Create customizations in the AWS Amazon Q console

4. How secure is the transformation process?

Amazon Q follows AWS security best practices:

- Your code is processed within secure AWS environments.
- Data is not stored or reused unless explicitly permitted.
- Integration with AWS IAM allows for fine-grained access control.
- Logs and artifacts can be stored securely in your AWS account.

For enterprise use, it's recommended to consult with your security team and AWS representative to understand compliance implications.

5. How do I provide feedback on Amazon Q suggestions?

You can:

- Use the thumbs up/down options in supported IDEs like VS Code.
- Submit feedback via the AWS Console when using Amazon Q there.
- Reach out through your AWS account team or solution architect if you are part of an enterprise engagement.

6. Does Amazon Q support cloud architecture suggestions or only code refactoring?

While its primary strength is in **code transformation and modernization**, Amazon Q can:

- Generate **cloud-native patterns**
- Assist in **infrastructure as code** generation (e.g., CDK, CloudFormation)
- Recommend **microservices architectures**

However, deeper architectural design decisions still require human review and cloud solution architect involvement.

7. Can Amazon Q modernize my entire application automatically?

Not fully. Amazon Q significantly **accelerates the modernization process** but doesn't eliminate the need for:

- Manual code reviews
- Custom business logic validation
- Integration and testing

It should be seen as an **AI-assisted co-pilot**, not a complete automation engine.

8. What IDEs or tools does Amazon Q integrate with?

Amazon Q integrates with:

- **VS Code**
- **JetBrains IntelliJ IDEA**
- **AWS Cloud9**
- **AWS Console**
- More integrations may be introduced in the future.

9. Where can I get help if I face issues?

- **AWS Support Center**: For paid support plans
- **AWS Developer Forums**: Community-driven help
- **GitHub Discussions**: For tool-specific questions (if open-sourced)
- **Internal Wiki / Accelerator Support Team**: If you're using Amazon Q as part of a company initiative

10. Can I use Amazon Q in air-gapped or restricted environments?

Currently, Amazon Q requires internet and AWS service access. It may not work in air-gapped or highly restricted enterprise environments unless AWS offers a private instance or region-specific support.
